Dead-Letter Queues

where messages go when they can't be processed

Watch the 4-min video

In one line: a dead-letter queue (DLQ) collects messages that keep failing so they stop blocking the main queue; set a max retry count, keep the original payload with error details, alert on DLQ growth, and have a tested way to fix and replay them.

1 The poison message

One malformed message arrives in your queue. The consumer tries to process it, crashes, and the message goes back to the queue. It's retried, it crashes again, and this loops forever. Meanwhile healthy messages behind it wait, or workers burn CPU on the same failure. That's a **poison message**.

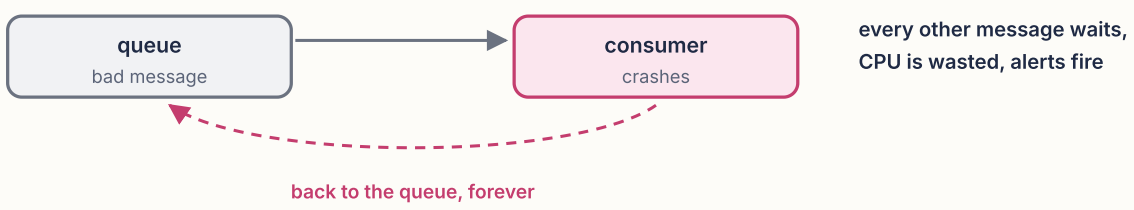


Figure 1. Without a limit, one bad message keeps failing forever.

2 What a DLQ does

A **dead-letter queue** is a separate queue or topic for messages that failed too many times. After N attempts the message is moved aside, the main flow continues, and someone can inspect the dead letters, fix the cause, and **replay** them. SQS, Azure Service Bus and RabbitMQ (dead-letter exchanges) support this natively; with Kafka, your consumer (or a framework like Spring Kafka or Kafka Connect) publishes failures to a `*.dlq` topic itself.

WORKED EXAMPLE: AN SQS REDRIVE POLICY

The orders queue has a dead-letter target and a max receive count of 5. A consumer receives a message, fails (missing customer ID), and doesn't delete it; after the visibility timeout it reappears. After the fifth receive, SQS moves it to `orders-dlq`. A CloudWatch alarm fires when the DLQ has any messages. The on-call engineer sees the error, deploys a fix that handles missing IDs, and uses SQS's redrive to move the messages back to the main queue, where they now succeed.

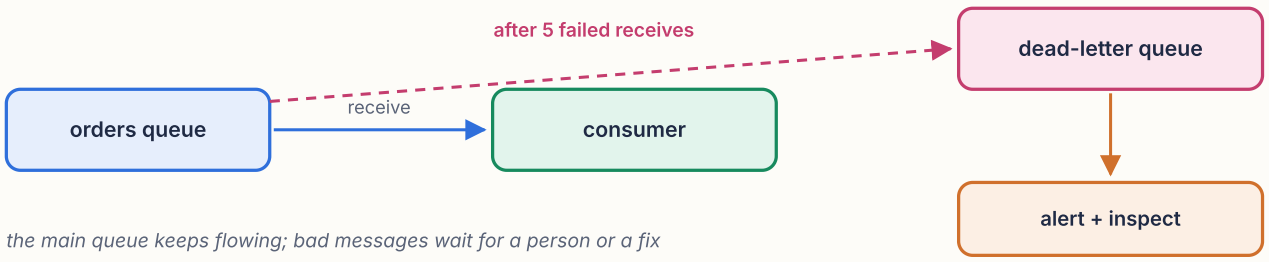


Figure 2. After five failed attempts, the message moves to the DLQ and an alert fires.

3 Doing it well

- **Separate failure types:** transient errors (timeouts) deserve retries with backoff (lesson 32); permanent ones (invalid data) can go to the DLQ immediately.
- **Keep context:** store the original message plus error details.
- **Alert** on DLQ depth and age; a DLQ nobody looks at is a data-loss bin.
- **Replay safely:** handlers must be idempotent, and replayed messages may now be out of order.
- Set the DLQ's retention longer than the main queue's.



Figure 3. What to keep with each dead letter.

COMMON MISTAKES

- No DLQ, so poison messages block or loop.
- A DLQ with no alert and no owner.
- Sending transient failures to the DLQ on the first error.
- Replaying a DLQ before fixing the bug, so everything fails again.

4 Key terms

Poison message: a message that always fails processing

Max receive count: attempts before dead-lettering

Visibility timeout: time a received SQS message is hidden from others

Dead-letter queue: holding area for failed messages

Redrive / replay: moving dead letters back for processing

5 Interview questions

Q1. What is a dead-letter queue for? BASIC

To move messages that repeatedly fail out of the main queue, so they stop blocking or looping, and keep them for inspection and replay.

Q2. When should a message go to a DLQ immediately? BASIC

When the error is permanent, like invalid format or a missing required field, so retries can't succeed.

Q3. Your Kafka consumer is stuck on one bad record and lag grows. What do you do? SCENARIO

Catch the error, publish the record with error metadata to a DLQ topic, commit past it, alert, and fix; for transient errors use retry topics with backoff.

Q4. Thousands of messages land in the DLQ after a deploy. How do you recover? SCENARIO

Roll back or fix the bug, verify on a few messages, then replay the DLQ in batches with idempotent handlers, watching for order-sensitive cases.

Q5. Why can a DLQ hurt ordering? TRICKY

A dead-lettered message is processed later than the ones after it, so per-key order breaks; strict cases must block the key or carry versions.

Q6. What's the risk of a DLQ with no monitoring? TRICKY

Messages silently pile up and expire after retention, which is data loss that nobody notices.

Next: SD-I-010 · Retries and Delayed Messages backendroadmap.com